

AUTHORED BY : ARPIT RAJ , LNMIIT JAIPUR

SQL Order By & Sorting Notes

By default, SQL does not guarantee the order of rows returned by a query.
If you want the results sorted, you must explicitly use the ORDER BY clause.

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary ASC;
```

```
SELECT *
FROM Employees
ORDER BY Salary DESC;
```

Multiple Column Sorting

```
SELECT *
FROM Employees
ORDER BY Department ASC,
Salary DESC;
```

SQL sorts in stages.

- **Step 1:** Sort by Department
- Within each department **Sort Salary descending.**

```
SELECT Name,
Price,
Quantity
FROM Products
ORDER BY Price * Quantity DESC;
```

```
SELECT FirstName,
LastName
FROM Students
ORDER BY CONCAT(FirstName, ' ', LastName);
```

```
SELECT  
Name,  
Salary * 12 AS AnnualSalary  
FROM Employees  
ORDER BY AnnualSalary DESC;
```

Handling NULLs in Sorting

Suppose

Name	Salary
Aadya	70000
Arpit	NULL
Riya	50000
Kabir	NULL

MySQL Ascending

```
ORDER BY Salary ASC;
```

Result

Name	Salary
Arpit	NULL
Kabir	NULL
Riya	50000
Aadya	70000

MySQL Workaround

To push NULLs last in ascending order

```
SELECT *  
FROM Employees  
ORDER BY Salary IS NULL,  
Salary;
```

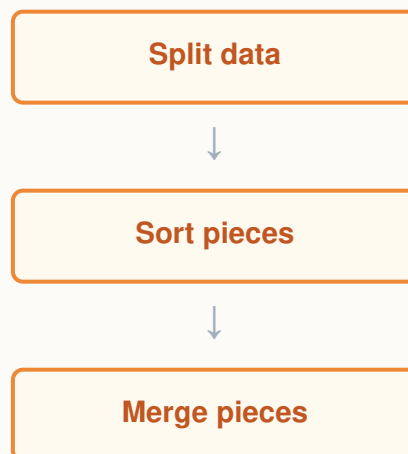
Salary IS NULL evaluates to:

- 0 for non-NULL
- 1 for NULL

So non-NULL values come first.

External Sorting

If the data doesn't fit into memory



This is why sorting huge tables can become slow.

With Index

Suppose

Index on Salary

- The rows are already organized.
- Database can often read them directly.
- No explicit sorting required.

- Much faster.

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 10;
```

Database doesn't always need to sort every row.
It can often stop after finding the top 10.